ELI — Voice Cloning Pipeline

ELI v2.1.93

August 2026

Contents

Voice Creation / Cloning Engine — end-to-end (2026-07-28)

How ELI turns a short audio/video clip of *any* voice — a character, an actor, a friend — into a usable ELI voice. Covers the full path from a dropped-in file to spoken output, the two distinct technologies involved, the dependency stack, and the real defects found and fixed while verifying it end-to-end on this machine.

The two voice technologies (don't confuse them)

ELI actually ships **two** different ways to get a "character" voice, and they trade off very differently:

	Character presets (`voice_fx.py`)	Voice cloning (`tts_xtts.py`)
Built-in	HAL, TARS, Rick, GLaDOS, JARVIS	none — always from *your* clip
How it works	base Piper voice + an ffmpeg `-af` effect chain (pitch/speed/filters)	Coqui XTTS-v2 zero-shot cloning from a ~6–20s reference clip
Fidelity	an approximation / vibe	reproduces the actual voice characteristics
Cost	free, instant, always available	opt-in ~1.8GB neural model, GPU preferred
Covers characters with no preset (Cartman, Vader, …)	no	yes

Cloning a real clip of HAL or Rick will sound closer to the source than the built-in ffmpeg preset — the presets exist so *something* is available with zero extra install; cloning is the accurate path once you have real audio.

End-to-end flow

```
user: "create a voice called vader from vader_clip.mp4"
  │        (or Settings ▸ Runtime ▸ "VOICE / TTS" ▸ drop-zone)
  ▼
router_enhanced.py  (voice.create pattern, ~line 1832)
  regex: (voice|clone) + (creat|make|build|clone|generat|record)
       + a media extension OR "from/using/with … recording/audio/clip/sample"
  extracts: name ("called X") + file (a .wav/.mp3/.mp4/.m4a/.ogg/.flac/.aac/.webm path)
  ──► {action: CREATE_VOICE, args: {name, file}}
  ▼
executor_enhanced.py  CREATE_VOICE handler (~line 10575)
  resolves ~ in the path, checks the file exists
  ──► tts_xtts.add_clone(name, path)
  ▼
tts_xtts.add_clone()                                    (offline, no TTS needed)
  ffmpeg -i <clip> -ac 1 -ar 22050 <ref.wav>   (mono/22.05kHz — any container, audio-only too)
  registry write: config/voices/clones.json   { name: {ref_wav, language, desc} }
  reference clip stored:                       models/voice_profiles/<name>.wav
  ──► {ok: true, id: "clone:<name>", synth_ready: xtts_available()}
  ▼
executor sets it active: tts_router.set_active_voice("clone:<name>")
  replies: "Created the voice '<name>' … set it active."
           (+ a note if the neural extra isn't installed yet)

── later, any time ELI speaks ──

tts_router.synthesize_wav(text, voice)
  voice startswith "clone:" ──► tts_xtts.synthesize_wav(text, name)
      _patch_transformers_compat()            (see "Defects found" below)
      os.environ["COQUI_TOS_AGREED"] = "1"    (see "Defects found" below)
      _get_model()  lazy-loads XTTS-v2 (~1.8GB, one-time download, cuda if free VRAM else cpu)
      model.tts_to_file(text, speaker_wav=ref.wav, language, file_path=tmp.wav)
      ──► WAV bytes
  None returned at ANY step ──► caller falls back to the normal active voice
      (never goes silent; CREATE_VOICE's own confirmation message says so up front)
```

Files involved

File	Role
`eli/execution/router_enhanced.py` (~1832)	`voice.create` pattern → `CREATE_VOICE`
`eli/execution/executor_enhanced.py` (~10570)	`CREATE_VOICE` handler → `add_clone` → sets active voice
`eli/perception/tts_xtts.py`	registry (`add_clone`/`get_clone`/`list_clones`/`delete_clone`), synthesis (`synthesize_wav`, `synthesize_natural_wav`), availability (`xtts_available`, `natural_available`)
`eli/perception/tts_router.py`	`synthesize_wav()` dispatches `clone:`/`natural:`/`char:`/plain Piper by prefix
`eli/perception/voice_fx.py`	the *other* path — ffmpeg character presets, unrelated dependency-wise
`config/voices/clones.json`	clone registry (name → reference wav path, language, description)

File	Role
`models/voice_profiles/<name>.wav`	the normalised 22.05kHz mono reference clip per clone

Dependency stack

The optional extra is declared three ways in `pyproject.toml` (`clone`, `natural`, `voice` — all aliases for the same thing, worded for whichever the user searches):

```
clone   = ["torch>=2.0", "torchaudio", "torchcodec", "coqui-tts>=0.24"]
natural = ["torch>=2.0", "torchaudio", "torchcodec", "coqui-tts>=0.24"]
voice   = ["torch>=2.0", "torchaudio", "torchcodec", "coqui-tts>=0.24"]
```

The torch trio is declared explicitly: `coqui-tts` does **not** declare torch/torchaudio/torchcodec itself, but imports all three at load time.

Installed from the repo root as `pip install -e ".[natural]"` (**not** `pip install "eli-v2.0[natural]"` — that string only resolves once the project itself is installed; it isn't published to PyPI under that name). On a box that already has `torch`+CUDA for the main model, only the delta installs.

In the shipped AppImage (from v2.1.65)

The `natural` extra is now bundled into the **Linux AppImage** — before that it was in neither `REQUIRED_EXTRAS` nor `OPTIONAL_EXTRAS`, so every shipped build registered cloned voices and then silently spoke in Piper. Three constraints shape how it is wired:

- **CPU torch only.** `release.yml` installs `torch torchaudio` from the pytorch CPU index *before* the extras loop; default PyPI torch drags ~6 GB of `nvidia-*` CUDA wheels in. A guard fails the build if any `nvidia-` package appears afterwards.
- **Linux only.** `OPTIONAL_EXTRAS` is shared by all three build jobs, so `natural` is a **job-level override on `build-linux`** — the only job that installs the CPU stack first. Adding it globally makes windows/macos resolve torch from PyPI and blows GitHub's 2 GiB per-asset limit, which fails the whole release.
- **Weights are not bundled.** The ~1.8 GB XTTS-v2 model downloads on first use. ELI is offline-by-default, so that first synthesis needs network access.

When the engine is absent the voice still registers, but synthesis falls back to Piper — and that fallback is now **visible** rather than silent: `tts_router` records what was requested and why it failed, logs it once per change, and the Settings ▸ Audio panel shows a `NOT IN USE` line naming the voice that is actually speaking. See `neural_fallback_state()`.

Defects found and fixed while verifying this end-to-end (2026-07-28)

Installing the extra and actually cloning a voice surfaced four real problems — none of them hypothetical, all reproduced and fixed on this machine:

1. **Non-editable install shadowed the source tree.** `pip install ".[natural]"` (no `-e`) builds and installs a frozen wheel copy of `eli-v2.0` into site-packages, with its own `eli`/`eli-gui`/etc. console scripts — silently diverging from the live checkout on every future edit. Fixed by uninstalling it and reinstalling with `-e`, matching what `install.sh` always does.
2. **`transformers` 5.x removed a function `coqui-tts` still imports.** `TTS.tts.layers.tortoise.autoregressive` imports `transformers.pytorch_utils.isin_mps_friendly`, which existed only as an old Apple-MPS `torch.isin()` workaround and is gone in `transformers>=5` — the exact version this project's own `requirements.lock.txt` pins for the rest of the stack (`transformers==5.8.1`), so downgrading transformers was not a safe option. Fixed with a lazy compat shim in `tts_xtts._patch_transformers_compat()`: if the name is missing, define it as plain `torch.isin()`

(identical behaviour off MPS). Called from both `_get_model()` (before loading) **and** `xtts_available()/natural_avai` — the first pass only patched the model-loading path, so the availability check itself still returned `False` even once cloning would actually have worked.

3. **`torchaudio/torchcodec` missing.** `coqui-tts` needs `torchaudio` (not auto-pulled here since a newer `torch` was already present) and, since `torch>=2.9` moved audio I/O off the old backends, `torchcodec` too. Installed `torchaudio==2.11.0+cu130` (matching the installed `torch 2.12.0+cu130`, via `--index-url https://download.pytorch.org/whl/cu130`) and `coqui-tts[codec]`.
4. **Interactive TOS prompt blocks first use with no TTY.** `coqui-tts` gates the *first* XTTS-v2 download behind `ModelManager.ask_tos()`, an `input()` y/n prompt ("I have purchased a commercial license" / "otherwise I agree to the non-commercial CPML"). ELI's GUI has no terminal for that prompt to read from — it would hang or raise `EOFError` on literally every user's first clone, GUI or voice command alike. Fixed in `_get_model()`: `os.environ.setdefault("COQUI_TOS_AGREED", "1")` before importing `TTS.api` — `ModelManager.tos_agreed()` short-circuits true on that env var, same effect as answering "y" at the prompt. This is always the non-commercial path here (a local, personal voice, never redistributed — same policy the shipped voice library already applies to restricted Piper voices).

Verified end-to-end (this session)

Ran the real path with no shortcuts — Piper-synthesised a reference clip (so the test needs no external/copyrighted audio), registered it exactly as `CREATE_VOICE` would, then synthesised *new* text (never in the reference) through `tts_xtts.synthesize_wav` directly (not the router's fallback-friendly wrapper, so a silent fallback to a normal voice could not masquerade as a working clone):

```
[1] Piper reference clip synthesised                       — 312,738 bytes
[2] add_clone("demo_clone", ref.wav)                        — {"ok": true, "synth_ready": true}
[3] xtts_available()                                        — True
[4] tts_xtts.synthesize_wav(new_text, "demo_clone")         — real XTTS-v2 clone, non-empty WAV
[5] tts_router.synthesize_wav(new_text, "clone:demo_clone") — same result through the full app path
```

First attempt (before fix #4) returned `None` at step 4 — the TOS prompt raised `EOFError`, and the *router* path (step 5) had silently produced normal Piper speech instead, which is exactly the failure mode a real user would have hit invisibly. Second attempt, with the fix in place, produced genuine cloned audio at both step 4 and step 5.

Licensing note

XTTS-v2 is Coqui's model under the **CPML (Coqui Public Model License)** — non-commercial personal use, which is exactly what this feature is for (a local, personal voice, never redistributed by ELI). A commercial deployment would need a paid licence from Coqui directly; that is a deployment decision, not something this pipeline enforces.

Known limitations

- First clone/natural use on a fresh install downloads ~1.8GB — no progress UI wired into the GUI yet beyond the console `tqdm` bar; a user driving this from chat/voice sees no feedback during the download.
- `synthesize_wav`/`synthesize_natural_wav` reload the model into the *same* process's GPU memory as the main GGUF model when both are active — no VRAM budget coordination between the two the way vision hot-swap has (`gguf_inference._LLM_CALL_LOCK`). Fine on this 24GB-class card; worth revisiting for 8GB targets if natural/clone voices see real usage there.
- No per-clone quality/duration validation on the reference clip — a 1-second or very noisy clip is accepted and will simply clone poorly rather than being rejected up front.